Cours MERN Stack – Groupe Décalé – Séance 6 (Finale)

Authentification Full-Stack JWT, Protection des Actions & Déploiement Cloud

Dr. Abdelweheb GUEDDES – École Polytechnique de Sousse – 2026–2027

Programme de la Dernière Séance (Fusionner Séance 3 + Séance 5 en Pratique) :

1. **Heure 1 (Dès la minute 5) :** On connecte React au serveur Express sécurisé par **Token JWT** (vu en Séance 3) et on crée le petit tiroir Redux `authSlice.js` (20 lignes).
2. **Heure 2 :** On ajoute l'**Intercepteur Axios** (5 lignes) qui envoie le badge `Bearer <token>` automatiquement et déverrouille la publication et la suppression d'articles.
3. **Heure 3 :** On prépare la **Mise en Production Cloud** (MongoDB Atlas + Render + Vercel) et le récapitulatif complet des 6 séances pour réussir votre projet MERN.

1 Étape 1 (Minute 5) : Le Serveur Express Protégé par Token JWT

1. Le Principe en Une Image : Visiteur (Lecture Seule) vs Membre Connecté (JWT)

En Séance 5, n'importe quel visiteur pouvait publier ou supprimer un article. Dans une vraie application, tout le monde peut **lire** les articles (`GET`), mais il faut présenter un **Token JWT valide** pour **ajouter** (`POST`) ou **supprimer** (`DELETE`) un article :



FIGURE 1 – Le fonctionnement complet de l'authentification JWT entre React et Express.

2. TP Immédiat 1 : Démarrer le Serveur Express Sécurisé (5 minutes)

Ouvrez un premier terminal dans le dossier `server/` et démarrez l'API :

```
cd server
npm install
npm start
```

Observez dans `server/server.js` comment le middleware `protect` (étudié en Séance 3) protège désormais les routes `POST` et `DELETE`, et comment le nom de l'auteur est lu directement depuis `req.user.name` :

```
// PUBLIQUE : Tout le monde peut lire les articles
app.get('/api/articles', (req, res) => res.status(200).json(articles));

// PROTEGEE PAR JWT : Le middleware "protect" verifie le Token avant d'autoriser l'ajout !
app.post('/api/articles', protect, (req, res) => {
  const newArticle = {
    id: Date.now().toString(),
    title: req.body.title.trim(),
    author: req.user.name, // Le nom de l'auteur vient directement du Token JWT !
    category: req.body.category || 'Architecture Web'
  };
  articles.unshift(newArticle);
  res.status(201).json(newArticle);
});
```

2 Étape 2 (Minute 20) : Créer le Tiroir `authSlice.js` en 20 Lignes (Redux + `localStorage`)

1. Pourquoi Combiner Redux Toolkit et `localStorage` ?

— **Redux Toolkit (`state.auth`) :** Permet à tous nos composants (`Navbar`, `App`, `ArticleCard`) de savoir instantanément qui est connecté pour afficher ou masquer les boutons !
— **`localStorage` :** Garde une copie du `user` et du `token` pour rester connecté même après `F5`.



FIGURE 2 – Quand l'utilisateur se connecte, `authSlice` met à jour l'écran (Redux) et sauvegarde le Token (`localStorage`).

2. TP Immédiat 2 : Créer `src/redux/authSlice.js` et l'Ajouter au Store

➥ Fichier `src/redux/authSlice.js` – exécuter avec `npm run dev`

```
import { createSlice } from '@reduxjs/toolkit';

const savedUser = localStorage.getItem('user');
const savedToken = localStorage.getItem('token');

const authSlice = createSlice({
  name: 'auth',
  initialState: {
    user: savedUser ? JSON.parse(savedUser) : null,
    token: savedToken || null
  },
  reducers: {
    // 1. Enregistrer l'utilisateur et son token JWT apres /api/auth/login
    loginSuccess: (state, action) => {
      state.user = action.payload.user;
      state.token = action.payload.token;
      localStorage.setItem('user', JSON.stringify(action.payload.user));
      localStorage.setItem('token', action.payload.token);
    },
    // 2. Deconnecter l'utilisateur et vider localStorage
    logout: (state) => {
      state.user = null;
      state.token = null;
      localStorage.removeItem('user');
      localStorage.removeItem('token');
    }
  }
});
export const { loginSuccess, logout } = authSlice.actions;
export default authSlice.reducer;
```

Puis ajoutez ce second tiroir dans `src/redux/store.js` : ➥ Fichier `src/redux/store.js` – exécuter avec `npm run dev`

```
import { configureStore } from '@reduxjs/toolkit';
import articleReducer from './articleSlice';
import authReducer from './authSlice';

export const store = configureStore({
  reducer: { articles: articleReducer, auth: authReducer }
});
```

3 Étape 3 (Heure 2) : L'Intercepteur Axios en 5 Lignes

1. Pourquoi l'Intercepteur Axios Change la Vie du Développeur

Sans intercepteur, il faudrait penser à écrire `headers: { Authorization: Bearer ${token}}` dans chaque bouton « Publier », « Supprimer », « Modifier »...

Avec un **Intercepteur Axios** ajouté une seule fois dans `src/api/axiosClient.js`, Axios vérifie automatiquement avant chaque départ de requête si un `token` est présent dans `localStorage` et l'attache tout seul !

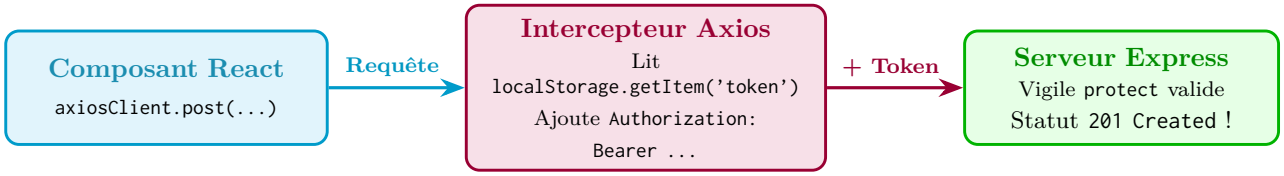


FIGURE 3 – Le guichet automatique de l'Intercepteur Axios : vos composants n'ont même plus à s'occuper du Token !

2. TP Immédiat 3 : Ajouter l'Intercepteur dans `src/api/axiosClient.js`

Ouvrez `src/api/axiosClient.js` et ajoutez les 6 lignes de l'intercepteur :

➥ Fichier `src/api/axiosClient.js` – exécuter avec `npm run dev`

```
import axios from 'axios';

// 1. Instance Axios pointant vers le serveur Express (ou l'URL Cloud en production)
const axiosClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api'
});

// 2. Intercepteur JWT : attache automatiquement le badge "Bearer <token>" a chaque requete
axiosClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default axiosClient;
```

Point de contrôle. Grâce à ces 6 lignes dans `axiosClient.js`, vos appels `axiosClient.post(...)` et `axiosClient.delete(...)` écrits en Séance 5 envoient désormais le Token JWT **automatiquement** sans changer une seule ligne dans vos composants !

4 Étape 4 : TP Immédiat 4 – Brancher la Carte de Connexion & Déverrouiller l'Interface

1. La Carte de Connexion (`src/components/AuthCard.jsx`)

Comme en Séance 5, la fonction `handleLogin` tient en **2 lignes** :

1. `const response = await axiosClient.post('/auth/login', { email, password });`
2. `dispatch(loginSuccess(response.data));`

➥ Fichier `src/components/AuthCard.jsx` – exécuter avec `npm run dev`

```
import React, { useState } from 'react';
import { useDispatch } from 'react-redux';
import axiosClient from '../api/axiosClient';
import { loginSuccess } from '../redux/authSlice';

export default function AuthCard() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errorMsg, setErrorMsg] = useState('');
  const [loading, setLoading] = useState(false);

  const dispatch = useDispatch();

  const handleLogin = async (e) => {
    e.preventDefault();
    setErrorMsg('');
    try {
      setLoading(true);
      // 1. Envoyer email + mot de passe au serveur Express
      const response = await axiosClient.post('/auth/login', { email, password });

      // 2. Ranger l'utilisateur et son Token JWT dans Redux + localStorage
      dispatch(loginSuccess(response.data));
    } catch (err) {
      setErrorMsg(
        err.response?.data?.message || 'Erreur de connexion au serveur Express.'
      );
    } finally {
      setLoading(false);
    }
  };

  const fillDemoAccount = () => {
    setEmail('gueddes@poly.tn');
    setPassword('123456');
    setErrorMsg('');
  };

  return (
    <section className="form-card auth-card">
      <div className="form-header">
        <span className="lock-pill">Espace Membre Securise (JWT)</span>
        <h3>Connexion Requise pour Publier</h3>
        <p>Connectez-vous pour obtenir votre Token JWT et debloquer l'ajout/suppression.</p
  >
      </div>

      {errorMsg && <div className="feedback-alert error">{errorMsg}</div>}

      <form onSubmit={handleLogin} className="article-form">
        <div className="form-group">
          <label htmlFor="email">Adresse Email :</label>
          <input
            id="email"
            type="email"
            placeholder="gueddes@poly.tn"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
          />
        </div>

        <div className="form-group">
          <label htmlFor="password">Mot de Passe :</label>
          <input
            id="password"
```

```
            type="password"
            placeholder="123456"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
          />
        </div>

        <button type="submit" className="btn-submit" disabled={loading}>
          {loading ? 'Verification...' : 'Se Connecter (Obtenir Token JWT)'}
        </button>

        <button type="button" className="btn-demo" onClick={fillDemoAccount}>
          Remplir Compte Demo (gueddes@poly.tn / 123456)
        </button>
      </form>
    </section>
  );
}
```

2. Déverrouiller l'Interface dans `App.jsx`, `Navbar.jsx` et `ArticleCard.jsx`

Dans **`src/App.jsx`**, une simple condition `{user ? <ArticleForm /> : <AuthCard />}` affiche le formulaire d'ajout si l'utilisateur est connecté, ou la carte de connexion s'il est visiteur :

➥ Fichier `src/App.jsx` – exécuter avec `npm run dev`

```
import React from 'react';
import { useSelector } from 'react-redux';
import Navbar from './components/Navbar';
import AuthCard from './components/AuthCard';
import ArticleForm from './components/ArticleForm';
import ArticleList from './components/ArticleList';

export default function App() {
  // On lit si un utilisateur est connecte dans le coffre Redux (state.auth)
  const { user } = useSelector((state) => state.auth);

  return (
    <div className="app-shell">
      <Navbar />
      <main className="container main-content">
        <div className="app-grid-layout">
          <div className="column-form">
            {/* Si connecte -> Formulaire d'ajout, sinon -> Carte de Connexion JWT */}
            {user ? <ArticleForm /> : <AuthCard />}
          </div>
          <div className="column-list">
            <ArticleList />
          </div>
        </div>
      </main>
    </div>
  );
}
```

Dans **`src/components/Navbar.jsx`**, on affiche le nom de l'utilisateur et le bouton « Déconnexion » (`dispatch(logout())`) :

```
const { user } = useSelector((state) => state.auth);
// ...
{user ? (
  <div className="user-badge connected">
    <span><strong>{user.name}</strong> ({user.role})</span>
    <button className="btn-logout" onClick={() => dispatch(logout())}>
      Deconnexion
    </button>
  </div>
) : (
  <div className="user-badge visitor">
    <span>Mode Visiteur (Lecture seule)</span>
  </div>
)}
```

Et dans **`src/components/ArticleCard.jsx`**, le bouton « Supprimer » n'apparaît que si `user` est connecté :

```
const { user } = useSelector((state) => state.auth);
// ...
{user ? (
  <button className="btn-delete" onClick={handleDelete}>Supprimer</button>
) : (
  <span className="locked-tag">Lecture seule (JWT requis)</span>
)}
```

5 Étape 5 : Résultat Visuel en Laboratoire & Déploiement Cloud

1. Les 3 États Visuels de Votre Application Full-Stack Sécurisée

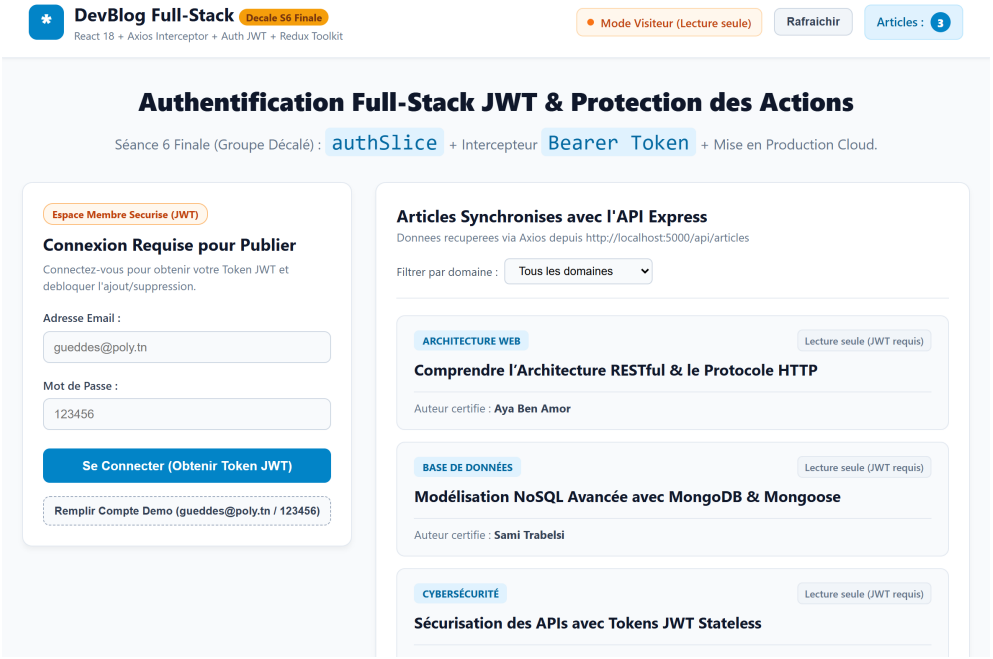
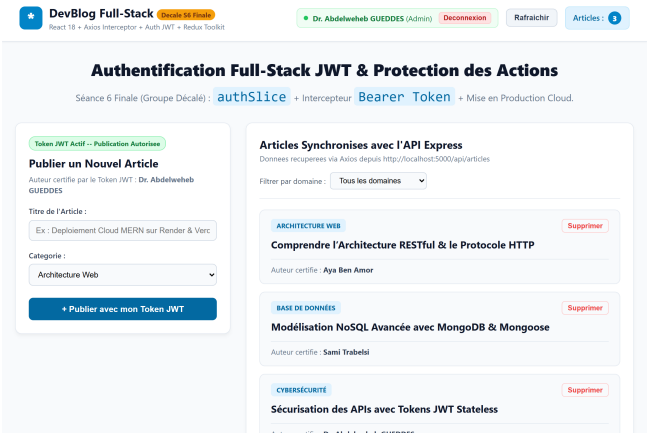
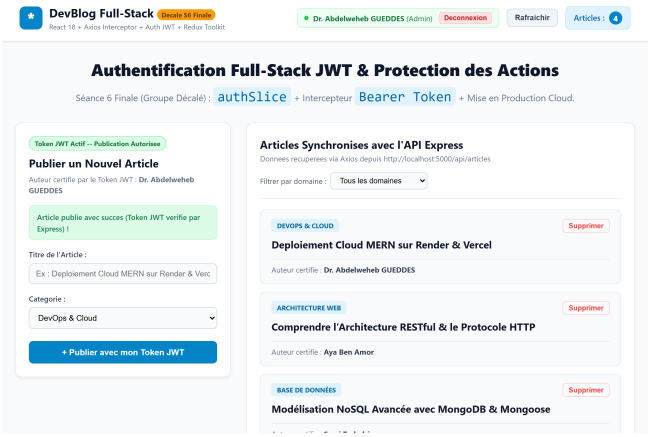


FIGURE 4 – **1. Mode Visiteur (Non connecté) :** Les articles publics s'affichent à droite en lecture seule, tandis que la colonne de gauche demande de se connecter.



(a) 2. Connexion JWT réussie : Le badge `Dr. Abdelweheb GUEDDES (Admin)` s'affiche en haut, le formulaire d'ajout et les boutons « Supprimer » se débloquent.



(b) 3. Publication avec Token JWT : L'intercepteur Axios envoie le token et Express certifie automatiquement le nom de l'auteur (`req.user.name`).

FIGURE 5 – Fonctionnement de bout en bout de l'authentification JWT avec Redux Toolkit et Axios.

2. Comment Déployer Votre Projet MERN sur le Cloud – Guide Pas à Pas

En laboratoire, les 3 briques tournent sur votre PC (`localhost:5173`, `localhost:5000`). En production, chaque brique part chez un hébergeur gratuit spécialisé. **L'ordre est imposé par les adresses** : l'API a besoin de l'adresse de la base, et le front-end a besoin de l'adresse de l'API. On déploie donc **de droite à gauche** : Base → API → Front-End.

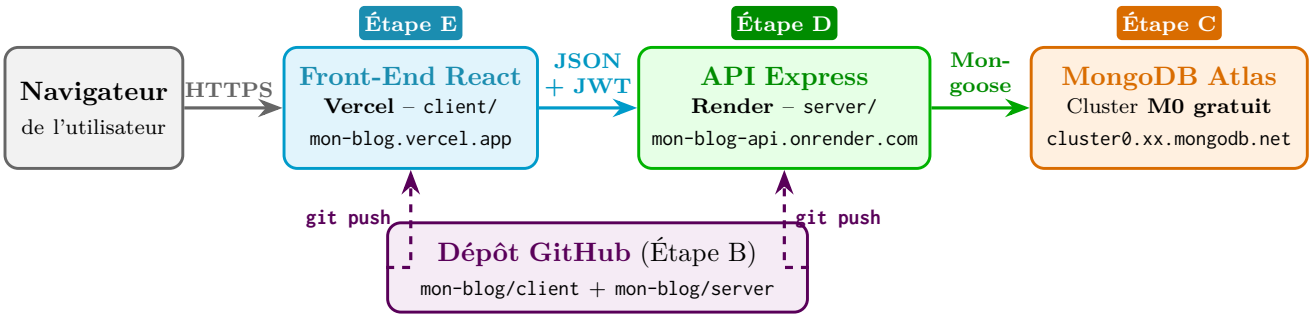


FIGURE 6 – Architecture de production : GitHub alimente Vercel et Render ; chaque `git push` redéploie automatiquement les deux.

Étape	Durée	Résultat obtenu à la fin de l'étape
A. Préparer le code (en local)	10 min	Le code lit ses adresses et secrets dans des variables d'environnement.
B. Publier sur GitHub	10 min	Le dépôt `mon-blog` contient `client/` et `server/`, sans `.env`.
C. Base MongoDB Atlas	15 min	Vous possédez une URI `mongodb+srv://...` testée en local.
D. API sur Render	15 min	`https://mon-blog-api.onrender.com/api/articles` renvoie du JSON.
E. Front-End sur Vercel	10 min	`https://mon-blog.vercel.app` affiche l'interface.
F. Fermer la boucle CORS	5 min	La connexion JWT, l'ajout et la suppression fonctionnent en ligne.

Étape A – Préparer le Code pour la Production (en local, 10 min)

Un hébergeur impose son port, ses adresses et ses secrets. Le code ne doit donc contenir **aucune valeur écrite en dur** : il lit tout dans `process.env` (serveur) ou `import.meta.env` (client).

A.1 – Le serveur lit son port et ses secrets dans l'environnement (`server/server.js`) :

```
const PORT = process.env.PORT || 5000;     // Render fournit son propre PORT
const JWT_SECRET = process.env.JWT_SECRET; // secret saisi sur Render, pas dans le code
// Production : CLIENT_URL = adresse Vercel. Local : toutes les origines.
app.use(cors({ origin: process.env.CLIENT_URL || '*' }));
```

A.2 – Le client lit l'adresse de l'API dans l'environnement (`client/src/api/axiosClient.js`) :

```
const axiosClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api'
});
```

A.3 – Le serveur possède un script `start` (`server/package.json`), car Render lance `npm start` :

```
"scripts": { "start": "node server.js", "dev": "node --watch server.js" }
```

A.4 – Le client compile sans erreur. Vercel exécutera exactement cette commande : si elle échoue chez vous, elle échouera en ligne.

```
cd client
npm run build      # crée le dossier client/dist/ (fichiers HTML/JS/CSS optimisés)
npm run preview    # sert dist/ sur http://localhost:4173 pour vérifier le résultat
```

Point de contrôle. `npm run build` affiche `built in ...ms` sans erreur, et l'application servie par `npm run preview` charge les articles depuis `localhost:5000`.

Étape B – Publier le Code sur GitHub (10 min)

Render et Vercel ne lisent pas votre disque : ils téléchargent le code depuis GitHub. Le dépôt contient les deux dossiers côte à côte (*monorepo*).

B.1 – Créer le fichier `.gitignore` à la racine du projet pour exclure les dépendances, les secrets et le build :

```
node_modules/
.env
dist/
```

B.2 – Créer un dépôt vide `mon-blog` sur `https://github.com/new` (sans README), puis pousser le code :

```
cd mon-blog                    # dossier qui contient client/ et server/
git init
git add .
git commit -m "Projet MERN pret pour le deploiement"
git branch -M main
git remote add origin https://github.com/VOTRE_PSEUDO/mon-blog.git
git push -u origin main
```

Sécurité. Le fichier `.env` ne doit **jamais** apparaître sur GitHub : il contient votre mot de passe Atlas et votre secret JWT. Si vous l'avez poussé par erreur, changez immédiatement le mot de passe Atlas et le `JWT_SECRET`, car l'historique Git conserve l'ancienne version.

Point de contrôle. Sur la page GitHub du dépôt, vous voyez `client/`, `server/` et `.gitignore`, mais ni `node_modules/` ni `.env`.

Étape C – Créer la Base de Données sur MongoDB Atlas (15 min)

Remarque. Le serveur du TP de cette séance stocke les articles dans un tableau en mémoire : il se déploie sans Atlas. L'étape C concerne votre **projet final**, qui utilise Mongoose (Séance 2). Sans base Cloud, toutes les données disparaissent à chaque redémarrage du serveur Render.

C.1 Créer le cluster. Sur `https://www.mongodb.com/cloud/atlas`, créez un compte, puis *Create Cluster* → offre **M0 (Free)**, fournisseur **AWS**, région **Europe (Frankfurt)**. Choisir la même région que Render réduit le temps de réponse.

C.2 Créer l'utilisateur de la base. Menu *Security* → *Database Access* → *Add New Database User* : identifiant `etudiant`, mot de passe composé **uniquement de lettres et de chiffres**, rôle *Read and write to any database*. Les caractères `@ : / ?` cassent l'URI de connexion.

C.3 Autoriser Render à se connecter. Menu *Network Access* → *Add IP Address* → *Allow Access from Anywhere* (`0.0.0.0/0`). L'offre gratuite de Render ne possède pas d'adresse IP fixe : sans cette règle, Atlas refuse la connexion.

C.4 Copier l'URI. *Clusters* → *Connect* → *Drivers*. Copiez l'URI, remplacez `<db_password>` par le mot de passe et ajoutez le nom de la base (`blog`) avant le `?` :

```
# server/.env (local uniquement)
MONGODB_URI=mongodb+srv://etudiant:MotDePasse2026@cluster0.ab12c.mongodb.net/blog?
    retryWrites=true&w=majority
JWT_SECRET=une_longue_phrase_secrete_difficile_a_deviner
```

C.5 – Tester la connexion en local avant de déployer (code de connexion de la Séance 2) :

```
mongoose.connect(process.env.MONGODB_URI)
  .then(() => console.log('MongoDB Atlas connecté'))
  .catch((err) => console.error('Erreur MongoDB :', err.message));
```

Point de contrôle. `npm start` dans `server/` affiche `MongoDB Atlas connecté`, et un article créé apparaît dans *Atlas → Browse Collections*.

Étape D – Déployer l'API Express sur Render (15 min)

D.1 – Sur `https://render.com`, cliquez sur *Get Started* puis connectez-vous **avec GitHub**.

D.2 – Cliquez sur *New +* → *Web Service*, puis sélectionnez le dépôt `mon-blog`.

D.3 – Remplir le formulaire de configuration :

Champ Render	Valeur à saisir	Pourquoi
Name	`mon-blog-api`	Devient l'adresse `mon-blog-api.onrender.com`.
Language	`Node`	Render installe Node.js.
Branch	`main`	Chaque `git push` sur `main` redéploie l'API.
Region	`Frankfurt (EU Central)`	Même région que le cluster Atlas.
Root Directory	**`server`**	Render ne doit lire que le dossier du back-end.
Build Command	`npm install`	Installe `express`, `cors`, `jsonwebtoken`...
Start Command	`npm start`	Lance `node server.js`.
Instance Type	`Free`	Offre gratuite (suffisante pour la soutenance).

D.4 – Ajouter les variables d'environnement (section *Environment Variables*). Elles remplacent le fichier `.env`, absent de GitHub :

Key	Value	Remarque
`JWT_SECRET`	`une_longue_phrase_secrete...`	Obligatoire (signe les tokens).
`MONGODB_URI`	`mongodb+srv://etudiant:...`	URI de l'étape C.
`CLIENT_URL`	(laisser vide pour l'instant)	Rempli à l'étape F.

Ne créez **pas** de variable `PORT` : Render la fournit lui-même, et `process.env.PORT` la lit.

D.5 – Cliquez sur *Deploy Web Service*. Suivez l'onglet *Logs* : `Build successful`, puis `Your service is live`.

D.6 – Tester l'API dans le navigateur : ouvrez l'adresse `https://mon-blog-api.onrender.com/api/articles`.

Point de contrôle. Le navigateur affiche le tableau JSON des articles. Notez cette adresse : elle sert à l'étape E.

Mise en veille gratuite. Après 15 minutes sans requête, Render endort le serveur gratuit. La requête suivante attend environ 50 secondes le réveil. Le jour de la soutenance, ouvrez l'URL de l'API une minute avant de présenter.

Étape E – Déployer le Front-End React sur Vercel (10 min)

E.1 – Sur `https://vercel.com`, cliquez sur *Sign Up* → *Continue with GitHub*.

E.2 – Cliquez sur *Add New...* → *Project*, puis *Import* à côté du dépôt `mon-blog`.

E.3 – Remplir l'écran *Configure Project* :

Champ Vercel	Valeur à saisir	Pourquoi
Project Name	`mon-blog`	Devient `mon-blog.vercel.app`.
Root Directory	**`client`** (bouton *Edit*)	Vercel ne compile que le front-end.
Framework Preset	`Vite` (détecté automatiquement)	Choisit les bonnes commandes.
Build Command	`npm run build`	Même commande que l'étape A.4.
Output Directory	`dist`	Dossier produit par Vite.
Environment Variables	`VITE_API_URL = https://mon-blog-api.onrender.com/api`	Adresse de l'étape D, avec `/api` et sans `/` final.

E.4 – Cliquez sur *Deploy*. Après environ une minute, Vercel affiche *Congratulations* et l'adresse `https://mon-blog.vercel.app`.

Repère Simple : Variable `VITE_` = valeur figée au build. Vite recopie `VITE_API_URL` **dans le JavaScript compilé** au moment de `npm run build`. Si vous modifiez cette variable sur Vercel, vous devez relancer un build : onglet *Deployments* → ... → *Redeploy*. Pour la même raison, une variable `VITE_` est lisible par tous : n'y placez jamais de secret.

Étape F – Fermer la Boucle CORS et Tester en Ligne (5 min)

À cet instant, l'interface s'affiche, mais le navigateur peut bloquer les appels vers Render : l'API doit déclarer que l'origine `mon-blog.vercel.app` est autorisée.

F.1 – Sur Render, ouvrez le service `mon-blog-api`, puis l'onglet *Environment*. Donnez à `CLIENT_URL` la valeur `https://mon-blog.vercel.app` (en `https`, **sans** `/` final), puis cliquez sur *Save Changes*. Render redéploie automatiquement l'API.

F.2 – Dérouler le scénario de test complet sur `https://mon-blog.vercel.app` :

1. Les articles s'affichent en mode visiteur : la route publique `GET /api/articles` répond.
2. La connexion avec votre compte affiche le badge dans la `Navbar` : la route `/api/auth/login` renvoie un token.
3. La publication d'un article réussit : l'intercepteur envoie le token et `protect` l'accepte.
4. Après `F5`, vous restez connecté : `authSlice` relit le token dans `localStorage`.
5. Dans `F12` → *Network*, la requête `POST` part vers `onrender.com` avec l'en-tête `Authorization: Bearer eyJ...`.

F.3 – Mettre à jour le site en ligne. Modifiez le code, puis `git add .`, `git commit -m "..."` et `git push` : Render et Vercel détectent le push et redéploient les deux briques en 1 à 3 minutes.

Dépannage : les 9 Erreurs les Plus Fréquentes

Symptôme observé	Cause	Solution
Console : `blocked by CORS policy`	`CLIENT_URL` différent de l'adresse Vercel (`/` final, `http`).	Corriger `CLIENT_URL` sur Render (étape F.1).
Requêtes vers `localhost:5000`, erreur `ERR_CONNECTION_REFUSED`	`VITE_API_URL` absente lors du build Vercel.	Ajouter la variable, puis *Redeploy*.
L'API répond `404` sur toutes les routes	`VITE_API_URL` saisie sans `/api`.	Valeur : `https://...onrender.com/api`.
Render : `Missing script: "start"` ou `Cannot find module`	*Root Directory* vide : Render lit la racine du dépôt.	*Settings* → *Root Directory* = `server`.
Render : `No open ports detected`	Port écrit en dur dans `app.listen(5000)`.	Utiliser `process.env.PORT || 5000`.
Logs : `MongoServerSelectionError`	Adresse IP de Render refusée par Atlas.	*Network Access* : ajouter `0.0.0.0/0`.
Logs : `bad auth : authentication failed`	Mot de passe erroné ou contenant `@ : / ?`.	Régénérer un mot de passe en lettres et chiffres.
Toutes les actions renvoient `401` après un redéploiement	`JWT_SECRET` modifié : les anciens tokens sont invalides.	Se déconnecter, puis se reconnecter.
Vercel : `404` au rafraîchissement d'une sous-page	Projet avec React Router : Vercel cherche un fichier inexistant.	Ajouter `client/vercel.json` (ci-dessous).

➥ Fichier `client/vercel.json` – uniquement si votre projet utilise React Router

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Checklist de soutenance.

- ☐ Les 3 adresses (GitHub, Vercel, Render) figurent dans le `README.md` du dépôt.
- ☐ Le fichier `.env` est absent de GitHub ; les secrets sont saisis sur Render et Vercel.
- ☐ Le scénario F.2 (lecture, connexion, publication, `F5`, suppression) réussit en ligne.
- ☐ L'API a été réveillée une minute avant la présentation.
- ☐ Un compte de démonstration est prêt pour le jury (email + mot de passe).

Les 3 Petits Défis Pratiques de Clôture & Bilan des 6 Séances

Défi 1 : Tester le 2e Compte Auteur (`aya@poly.tn`) ★

Cliquez sur « Déconnexion » dans la `Navbar`, puis connectez-vous avec `aya@poly.tn` (mot de passe : `123456`) et publiez un article. Vérifiez que l'article publié porte automatiquement la signature « **Auteur certifié : Aya Ben Amor** » grâce au Token JWT !

Défi 2 : Basculer entre « Connexion » et « Inscription » dans `AuthCard.jsx` ★★

Dans `AuthCard.jsx`, ajoutez un state `const [isRegister, setIsRegister] = useState(false);` et un champ `name` affiché quand `isRegister` vaut `true` pour appeler la route d'inscription :

```
const response = await axiosClient.post('/auth/register', { name, email, password });
dispatch(loginSuccess(response.data));
```

Défi 3 : Générer le Build de Production React (`npm run build`) ★★★

Dans le terminal `client/`, lancez `npm run build` et vérifiez que le dossier optimisé `client/dist/` est généré en moins d'une seconde avec 0 erreur !

Récapitulatif Général de Votre Parcours MERN (Groupe Décalé – Séances 1 à 6)

Séance	Compétence Clé Acquise	Fichiers / Outils Maîtrisés
Séance 1	Serveur Express & Architecture SoC	`server.js`, `routes/`, `controllers/`, `req.body`, statuts HTTP.
Séance 2	Base Cloud MongoDB Atlas & Mongoose	`mongoose.Schema`, `async/await`, `find()`, `create()`, `findByIdAndDelete()`.
Séance 3	Sécurité Back-End JWT & Bcrypt	Hachage `bcrypt.hash`, jeton `jwt.sign`, middleware sentinelle `protect`.
Séance 4	Front-End React 18, Vite & Hooks	Composants JSX, `props`, `useState`, `useEffect`, formulaires contrôlés.
Séance 5	Communication Axios & Redux Toolkit	`axiosClient.get/post/delete`, `createSlice`, `useSelector`, `useDispatch`.
Séance 6	Auth Full-Stack JWT & Déploiement Cloud	`authSlice` + `localStorage`, Intercepteur `Bearer Token`, `npm run build`, Atlas + Render + Vercel.